

- `<s:Rect width="100%" height="100%">`
- `<s:fill>`
- `<s:SolidColor color="#cccccc"/>`
- `</s:fill>`
- `</s:Rect>`

Add another rectangle after the first, but still before the `contentGroup` object. This rectangle is constrained to 20 pixels from all application borders, uses rounded corners, and has a drop-shadow filter:

- `<s:Rect left="20" right="20" top="20" bottom="20"`
- `radiusX="15" radiusY="15">`
- `<s:fill>`
- `<s:SolidColor color="#eeeeee"/>`
- `</s:fill>`
- `<s:filters>`
- `<s:DropShadowFilter/>`
- `</s:filters>`
- `</s:Rect>`

The custom skin now creates a framed appearance. Next you'll add settings to ensure that visual objects that are added to the application from the host component are placed within the visual area created by the frame. You do this by adding constraint properties to the `contentGroup` object in the custom skin. Now, add `left`, `right`, `top`, and `bottom` properties to the `contentGroup` object, setting all values to 30. This causes any visual objects added to the Application to be placed within the rounded rectangle with extra padding of 10 pixels on all borders:

- `<s:Group id="contentGroup"`
- `left="30" right="30" top="30" bottom="30"/>`

Binding a custom skin to a Spark component

You bind a custom skin to a Spark component using the component's `skinClass` style. This style is declared in the `SkinnableComponent` class and is therefore available to all of its subclasses. The `skinClass` style's type is set to `Class`, but the class must extend `UIComponent`. Skins designed for use with Spark components should extend `SparkSkin`. As with all styles, you can set the `skinClass` style in many ways. You can set it as an inline declaration for a particular component instance or in a style sheet to affect all instances of a particular component or at runtime using the `setStyle()` method.